

A Third Executor: Tapping the Integrated GPU via OpenCL

Matias Saavedra

Tuesday 7th July, 2026

Binary: branch feat/igpu-opencl (intended tag binary-v6-igpu) built on top of binary-v5-affinity (fc33e29). New translation unit oclkernel.cpp/.h; the CUDA and CPU paths are unchanged. Baseline is the same binary in gpuMode=1 (discrete GPU only).

Resumen

The renderer used two executor classes — the discrete GPU (CUDA) and a pool of CPU worker threads. This report adds a **third**: an OpenCL backend (oclkernel.cpp) that runs the FP64 Mandelbrot kernel on the *integrated* GPU (Intel UHD 630, on the i7-9750H die). `compute()` now dispatches by an `ExecKind` enum, both GPU backends share a result-copy helper, and the iGPU runs on its own `CalcThr` with the same largest-first affinity as the dGPU. Output is **equivalent to the CUDA path to within FP rounding** (identical on shallow frames; at most 0.149% of pixels per frame differ on deep frames, 0.050% over the run — full-run verification in S2.3). The headline A/B (100 frames, 1920×1080 , `diffT`=0.1, `save`=1, 12 threads, 3 reps, on AC) measured a **wall win, not the predicted wash: dGPU+iGPU+10CPU 44.32 s vs. dGPU+11CPU 57.72 s, -23.2%** (disjoint ranges; -72.6% vs. the 161.6 s CPU floor) — and it wins *while surrendering a CPU worker*. The mechanism is not the predicted “GPU saturated”: the binding lane is the CPU pool (busiest thread $\approx$ wall in every config), and the second GPU strips total CPU compute from 622 s to 435 s (-30%), faster than removing a CPU thread raises per-thread load. The two GPUs self-balance (34.4 vs. 30.4 s kernel).

1. Setup and the Change Measured

Workload and machine.

As in reports 12 and 13: the same `spec.in` (100 frames, 1920×1080 , zoom to `maxIterations` = 10,000), i7-9750H (6c/12t) + GTX 1660 Ti Max-Q (*discrete* GPU, CUDA) + Intel UHD 630 (*integrated* GPU, OpenCL), 12 threads, `diffThreshold` = 0.1, `pixelSizeThresh` = 32,768, `save` = 1, on AC power. The decomposition is unchanged from binary-v5-affinity: **5,608 leaves** in every configuration — only *which* processor computes each leaf differs.

Three executors, one dispatch point.

The worker model was binary (GPU thread vs. CPU thread). It becomes a three-way `ExecKind`; the work queue still distinguishes only GPU-class (largest-first) from CPU (smallest-first), so `WorkQueue` is untouched. The branch lives entirely in `compute()`’s dispatch and the new translation unit:

Código 1: mandelregion.cpp:187–193 — compute() picks a front-end by executor kind.

```

1 if (kind == EXEC_CUDA)
2   hostFE (upperX, upperY, lowerX, lowerY, pixelsX, pixelsY, &h_res, &pitch,
   ↪ MAXGRAY);
3 else // EXEC_IGPU
4   oclFE (upperX, upperY, lowerX, lowerY, pixelsX, pixelsY, &h_res, &pitch,
   ↪ MAXGRAY);
5
6 commitGPUResult (h_res, pitch / sizeof (int),
7                  kind == EXEC_CUDA ? "GPU" : "iGPU");

```

oclFE mirrors the CUDA hostFE signature exactly, so the only difference between the two GPU paths is the function called. Both then run the identical copy-and-metrics loop, factored out of the old CUDA branch into commitGPUResult (so the discrete path is byte-for-byte the same operations it was, only relabelled):

Código 2: mandelregion.cpp:137–160 — the shared GPU result-copy, used by both backends.

```

1 void MandelRegion::commitGPUResult (unsigned int *h_res, int pitchInts,
2                                     const char *backend) {
3   // ... for each pixel: read h_res[j*pitchInts + i], accumulate iter/inset
4   //   metrics, write colormap[color] (or black for in-set) into the QImage.
5   //   Identical to the previous inline CUDA copy; only the label differs.
6 }

```

The OpenCL backend (oclkernel.cpp).

Compiled by g++ (not nvcc) against CUDA’s bundled CL headers and linked through the ICD loader (-lOpenCL). It exposes three calls that parallel kernel.cu: OCLmemSetup / oclFE / OCLmemCleanup. The kernel is a line-for-line port of mandelKernel, embedded as a string so there is no .cl file to locate at runtime:

Código 3: oclkernel.cpp:62–85 — the FP64 Mandelbrot kernel, embedded.

```

1 #pragma OPENCL EXTENSION cl_khr_fp64 : enable
2 int diverge(double cx, double cy, int MAXITER) {
3   int iter = 0; double vx = cx, vy = cy, tx, ty;
4   while (iter < MAXITER && (vx*vx + vy*vy) < 4.0) {
5     tx = vx*vx - vy*vy + cx; ty = 2.0*vx*vy + cy; vx = tx; vy = ty; iter++;
6   }
7   return iter;
8 }
9 __kernel void mandelKernel(__global unsigned int *out, double upperX, double
   ↪ upperY,
10                            double stepX, double stepY, int resX, int resY, int MAXITER) {
11   int myX = get_global_id(0), myY = get_global_id(1);
12   if (myX >= resX || myY >= resY) return;
13   out[myY*resX + myX] = (unsigned)diverge(upperX + myX*stepX, upperY - myY*
   ↪ stepY, MAXITER);
14 }

```

oclFE sets the kernel arguments, launches a 2D NDRange rounded up to whole 16×16 work-groups (the in-kernel bounds check drops the overhang, exactly as the CUDA grid/block guard does), reads the region back, and times the kernel and the read with OpenCL profiling events — the direct analogue of the CUDA-event timing in hostFE:

Código 4: oclkernel.cpp:264–271 — launch + readback, device-timed by events.

```
1 clEnqueueNDRangeKernel(queue, kernel, 2, NULL, global, local, 0, NULL, &evKernel
   ↪ );
2 clEnqueueReadBuffer(queue, d_res, CL_TRUE, 0, resX*resY*4, h_res, 0, NULL, &
   ↪ evRead);
3 // CL_PROFILING_COMMAND_START/END on evKernel, evRead -> per-region
   ↪ kernel/read ms
```

Device selection and the per-thread context.

OCLmemSetup enumerates every OpenCL GPU device and picks the *integrated* one: it prefers an Intel-vendor GPU, accepts any non-NVIDIA GPU as a fallback (the discrete card is already driven through CUDA — running OpenCL on it too would be pointless contention), honours a MANDEL_OCL_DEVICE override, and aborts with install guidance if none is found. Because an OpenCL context is best used from one thread, the iGPU worker builds and tears down its own context, queue, program and buffers *inside its own run()* — file-scope state touched by exactly one thread, the same single-thread assumption kernel.cu makes:

Código 5: main.cpp:172–189 — the iGPU worker owns its OpenCL state for its lifetime.

```
1 if (kind == EXEC_IGPU) OCLmemSetup (resX, resY);    // build context on this
   ↪ thread
2 bool isGPU = (kind != EXEC_CPU);                    // both GPU classes pull
   ↪ largest
3 while ((t = que->extract (isGPU)) != NULL) { t->examine (*que, kind); delete t; }
4 MandelRegion::printCPUSummary();
5 if (kind == EXEC_IGPU) OCLmemCleanup ();
```

The CLI.

arg3 becomes a backend bitmask, gpuMode: bit 0 = discrete GPU (CUDA), bit 1 = integrated GPU (OpenCL). So 0 = CPU only, 1 = dGPU (the historical default, preserved), 2 = iGPU, 3 = both. In dual mode thread 0 is the dGPU (main thread, wrapped by CUDAmemSetup/Cleanup) and the iGPU runs on worker thread 1.

The runtime gotcha.

The UHD 630 (Coffee Lake, Gen9.5, 8086:3e9b) is *not* served by the current intel-compute-runtime (NEO $\geq 24.x$ dropped Gen8–Gen11; it loads but prints FATAL: Unknown device). The Gen9.5 part needs the *legacy* runtime (intel-compute-runtime-legacy1 on Arch). With it installed, clinfo reports the device with cl_khr_fp64 present, so the double-precision kernel builds.

Measurement.

A four-config wall A/B at 12 threads, 3 reps each, all on one binary in one thermal state so the comparison is disjoint (the discrete-GPU baseline is re-measured here, not reused from reports 05/12).¹ Raw data: `experiments/20-igpu-openc1/`.

2. Results

2.1. Wall Time — The Win

Tabla 1: Wall time by backend mode (12 threads, `diffT` = 0.1, `save` = 1, 3 reps). Δ is relative to `dGPU+11CPU`, the prior best. The mode-1 and mode-3 ranges are *disjoint* (every mode-3 rep beats every mode-1 rep), so -23.2% is signal, not noise.

Config	Mode	Wall mean (s)	Range (s)	Δ vs. mode 1
CPU12	0	161.64	159.87–163.03	+103.92 (+180%)
iGPU+11CPU	2	81.72	81.06–82.75	+24.00 (+41.6%)
dGPU+11CPU	1	57.72	57.34–58.01	— (baseline)
dGPU+iGPU+10CPU	3	44.32	44.14–44.62	−13.40 (−23.2%)

¹ The `save=1` PNG writes raise the absolute wall ~ 5 s above the `save=0` numbers of reports 08–13, so the 57.72 s baseline here is *not* comparable to report 12’s 49.86 s. A separate `save=0` cross-check (`experiments/20-igpu-openc1/ab_save0.csv`; alternating mode 1/mode 3, 3 reps) confirms the gap is `save` + thermal, not a regression: it reproduces report 12’s `save=0` baseline at mode 1 (**49.24 s** vs. 49.86 s) and gives mode 3 at **38.19 s** (-22.4% , a new best, -23.4% below report 12). The win is identical in both regimes; the CUDA path is byte-identical to `binary-v5-affinity`.

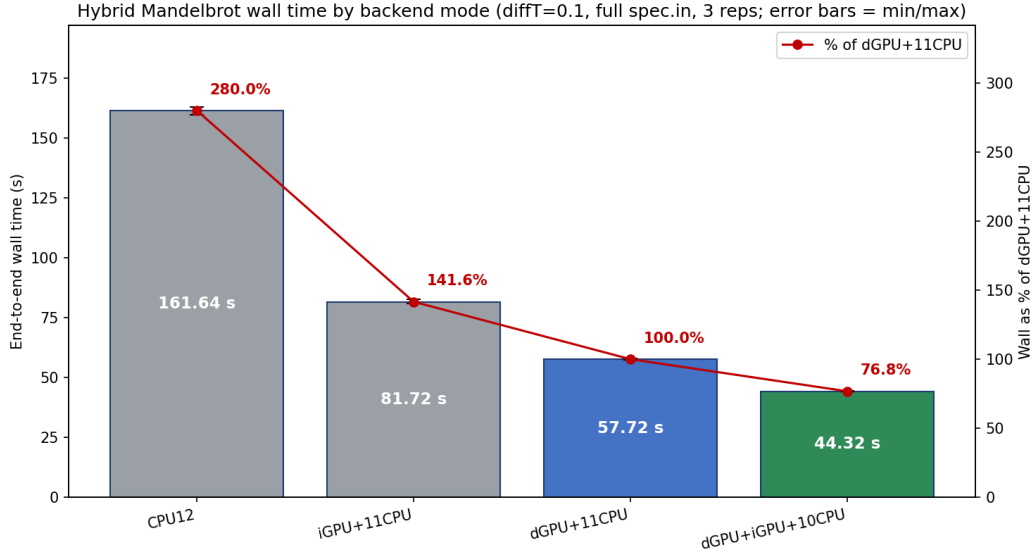


Figure 1: End-to-end wall by backend mode (bars, left axis; red line = % of the dGPU+11CPU baseline, right axis). The green bar is the new configuration: adding the iGPU drops the wall to 76.8% of the current best. Error bars are min/max over 3 reps — invisibly small. Note the iGPU *alone* (mode 2) already halves the CPU floor.

2.2. Where the 5,608 Leaves Ran

Tabla 2: Per-executor breakdown (rep 1; the reps agree to < 1%). Leaves sum to 5,608 in every row (decomposition-identical). “CPU/thread” is total CPU compute over the CPU-thread count; it tracks the wall to ~98%, identifying the CPU pool as the binding lane.

Config	dGPU (reg/s)	iGPU (reg/s)	CPU reg	CPU thr·s	CPU/thr (s)	out- liers
CPU12	0	0	5,608	1,898	158.2	4
iGPU+11CPU	0	2,960 / 65.8	2,648	898	81.6	0
dGPU+11CPU	3,447 / 42.9	0	2,161	622	56.6	0
dGPU+iGPU+10CPU	2,915 / 34.4	1,422 / 30.4	1,271	435	43.5	0



Figure 2: The 5,608 leaves split by executor class. Every bar reaches the same dashed total (the decomposition never changes); only the assignment shifts. In the winning mode 3 the CPU pool (orange) is squeezed to 1,271 leaves — the two GPUs between them absorb 4,337 — and the iGPU (green) claims 1,422, a third of the GPU work.

2.3. Correctness

A full-run verification — all 100 frames rendered three times, every pixel by the CUDA path (**mode 1**, 1 thread), every pixel by the OpenCL path (**mode 2**, 1 thread), and every pixel by the CPU path; script and per-frame data in `experiments/20-igpu-openccl/verify_output.{sh,csv}` — *corrects* an earlier single-frame check that had overgeneralised: the two GPU backends are **not byte-identical in general**. They agree exactly on the two cheap wide frames (0–1) and differ on every deeper frame by a small, depth-growing set of pixels: **104,355 px over the whole run (0.050%** of 207.36M), rising monotonically from 2 px (frame 2) to a peak of 3,089 px (**0.149%**, frame 88). Monotonic growth with `maxIter` is the signature of accumulated rounding divergence — the two kernel compilers make different FMA-contraction choices in the iteration recurrence, flipping `diverge()` by ± 1 for orbits near the escape threshold.

Both GPUs differ from the *CPU* render two orders of magnitude more — **16.73M px (8.07%** of the run, up to 19.7% on the deepest frame) — and they do so near-identically: their vs-CPU counts track within a few pixels per frame (69 px summed over the run). The host build (`g++ -O2`, no FMA without `-mfma`) sits on one side of the contraction gap; both GPUs sit on the other. The practical statement: **the iGPU path is exactly as correct as the CUDA path** — the same ± 1 -iteration rounding class, no iGPU-specific artifact — but bitwise GPU–GPU identity holds only on shallow frames.

3. Analysis

3.1. The win is real, large, and disjoint

The mode-1 and mode-3 ranges do not overlap (Table 1: 57.34–58.01 s vs. 44.14–44.62 s, a ~ 13 s gap), and the within-config spread is under 0.5 s. The -23.2% is therefore unambiguous signal. It is the largest single-step wall improvement in the series — GPU affinity (`12-gpu-affinity.tex`) moved the wall -3.6% ; this moves it -23.2% — and it is achieved while the configuration *gives up a CPU worker* (10 vs. 11), so the integrated GPU is worth far more than the eleventh CPU thread it displaces.

3.2. The mechanism: the CPU pool is the binding lane, and the iGPU shrinks its work

The wall is the busiest CPU thread.

In every accelerated config the CPU pool is tightly balanced and each CPU thread is busy for essentially the whole run: total CPU compute over the CPU-thread count equals the wall to $\sim 98\%$ (Table 2, “CPU/thr” vs. Table 1: 56.6 vs. 57.7 s; 43.5 vs. 44.3 s; 81.6 vs. 81.7 s). So to first order wall $\approx$ (total CPU compute)/(#CPU threads).

Adding the iGPU lowers the numerator faster than the denominator.

Mode 3 removes one CPU thread (11 $\rightarrow$ 10, which alone would *raise* per-thread load by 10%) but the two GPUs together strip total CPU compute from **622 s to 435 s** (-30% , -187 s) by absorbing 1,166 more leaves off the pool (CPU leaves 2,161 $\rightarrow$ 1,271, Fig. 2). Net per-thread CPU load falls 56.6 $\rightarrow$ 43.5 s (-23%), and the wall follows. This is why the prediction failed (next subsection): the lever is CPU-work *removal*, and a second GPU removes more of it than a single GPU could.

Why one GPU could not do this alone.

Not because the dGPU declines work: `WorkQueue::extract` has no size threshold — a GPU-class thread unconditionally takes the largest pending region whenever it is free, and with all 100 frame roots queued up-front the queue is non-empty until the very tail. The limit is that a GPU worker is a *serial lane*: each region occupies the thread for the kernel *plus* its host-side stages — `examine()`’s stencil sampling, the synchronous copy, `commitGPUResult`’s per-pixel `setPixel` walk over the region (the dGPU commits the large-region majority of all ~ 207 M pixels), and its share of the `save=1` PNG encodes. The 42.9 s of mode-1 kernel time is only the *device-side* part of that lane; the other ~ 15 s of the 57.7 s wall is host-side lane work, not idleness. One lane’s end-to-end throughput therefore caps how many leaves the GPU class can absorb within the wall. Mode 3 adds a *second* serial lane, nearly doubling GPU-class throughput, and it is that added capacity — not any change in scheduling policy — that strips the extra 187 s off the CPU pool.

3.3. The prediction was wrong — and why

The pre-measurement call (recorded on the branch) was “marginal-to-negative: FP64-weak iGPU thermally coupled to the CPU pool, which is the binding wall.” The direction was right about *what* binds (the CPU pool) but wrong about the *consequence*. The error was assuming a third executor can only *rebalance* a binding CPU pool (which cannot help a balanced pool — the lesson of `11-priority-queue.tex`). In

fact the iGPU does not rebalance the CPU pool; it *removes work from it*, which is exactly the work-reduction lever that reports 09 and 12 named as the only way past the post-affinity floor. The feared thermal/bandwidth contention is real but second-order here: the dGPU is discrete silicon (uncoupled), and the iGPU’s contention with the CPU pool is dwarfed by the 187 s of CPU compute it offloads.

3.4. The two GPUs self-balance; the iGPU alone already halves the floor

Under largest-first affinity the two GPUs end mode 3 with **34.4 s (dGPU)** and **30.4 s (iGPU)** of kernel time — within 12% despite the iGPU’s $\sim 1.8\times$ higher per-region cost (21.4 vs. 11.8 ms/region) — because the faster device simply pulls more regions. Note that kernel time understates lane occupancy: adding each lane’s host-side commit work (previous subsection), both GPU threads finish close to the 44 s wall, i.e. mode 3 ends in a *three-way* balance (dGPU lane, iGPU lane, CPU pool), which is what a work-conserving queue should produce. Separately, the iGPU *as the sole accelerator* (mode 2) cuts the CPU-only floor nearly in half ($161.6 \rightarrow 81.7$ s, -49%) by absorbing the interior outliers: CPU outliers (>5 s regions) drop from 4 to 0 in every accelerated mode (Table 2), including the 13.5 s frame-73 minibrot interior that pins a CPU thread in mode 0.

4. Recommendations

1. **Adopt `gpuMode = 3` (both GPUs) as the default on this machine.** It is a clean -23.2% ($57.72 \rightarrow 44.32$ s) with disjoint A/B ranges, decomposition-identical, output equivalent to CUDA’s to within ± 1 -iteration FP rounding ($\mathcal{S}2.3$), and contained to one new translation unit. Merge `feat/igpu-opengl` and tag `binary-v6-igpu`.
2. **Try strict dGPU-priority-on-the-largest.** Both GPUs currently pull from the same end, so the weaker iGPU can grab the single biggest outlier the dGPU would clear in $\sim 1/1.8$ the time. Reserving the very largest region for the dGPU (iGPU takes second-largest) may shave the tail further; expected to be small (the GPUs already self-balance to 12%) but it is a pure scheduling change in `WorkQueue`.
3. **Widen both GPU lanes by cheapening the host-side commit.** The corrected mechanism above says GPU-class throughput is *lane-bound*, and a large share of each lane is `commitGPUResult`’s per-pixel `QImage::setPixel` walk. Replacing it with `scanLine()` row writes (one `QRgb*` per row, same pixels) is a small, local change that would let *both* GPU lanes absorb more leaves per second — the same lever as adding a lane, without new hardware. Worth measuring before the scheduling refinement of item 2.
4. **Run the full Fig. 11.15 thread-scaling sweep in mode 3.** This report fixed 12 threads; the dGPU-vs-dGPU+iGPU scaling curves (`GPU+kCPU`) would show where the iGPU’s contribution saturates as CPU workers are added, comparable to `experiments/05-fig1115-postfix`.

5. **Generalise off this laptop with care.** The win is conditional on the CPU pool being the binding lane at `diffT=0.1` with `save=1`; on a machine where the dGPU saturates first, a second weak GPU would help less. The mechanism (CPU-work removal), not the magnitude, is what transfers.

5. Conclusion

Adding an OpenCL backend for the integrated GPU — a third **ExecKind** that shares the dGPU’s largest-first affinity and the CPU’s work queue, dispatched at one point in `compute()` — cuts the end-to-end wall by 23.2% ($57.72 \rightarrow 44.32$ s, disjoint ranges), and by 72.6% against the CPU-only floor. The configuration wins despite trading a CPU worker for the iGPU thread. The mechanism is measured, not assumed: the CPU pool is the binding lane (busiest thread $\approx$ wall), and the second GPU strips total CPU compute by 30% ($622 \rightarrow 435$ s), more than offsetting the lost worker. This refutes the pre-measurement “marginal-to-negative” prediction, which mistook a work-removal lever for a rebalancing one. The two GPUs self-balance to within 12%, the iGPU output matches CUDA to within FP rounding (0.050% of pixels over the run, *S2.3*), and the iGPU alone halves the CPU floor. Scheduling was exhausted at -3.6% (*12-gpu-affinity.tex*); adding hardware that removes CPU work is what moved the wall again.